Inter-process Communication

Producer-Consumer Problem



By:

Dr. H. K. Azad
Assistant Professor (Sr)
SCOPE
Email: hiteshwarkumar.azad@vit.ac.in

Processes in a System

- Processes within a system may be **independent** or **cooperating**
- **Independent processes**:
  - Do not affect or get affected by other processes.
  - No data sharing.
- **Cooperating processes**:
  - Can affect or be affected by other processes.
  - Can share data and resources.
- **Reasons for Cooperation**
  - **Information Sharing**
    - Multiple users or applications may need access to the same data (e.g., shared files, databases).
  - **Computation Speedup**
    - Tasks can be divided among processes to execute concurrently (parallelism).
  - **Modularity**
    - Breaking a large system into smaller, manageable processes that interact.
  - **Convenience**
    - Easier for users when processes communicate and work together (e.g., a word processor and spell checker).

Interprocess Communication (IPC)

- Required when processes cooperate.
- Two fundamental models:

1. Shared Memory

- Region of memory shared by cooperating processes.
- Processes read/write to this shared space.
- Managed by **users/programmers**.
- Faster (no kernel involvement once set up).
- Example: Producer-Consumer problem using a buffer.

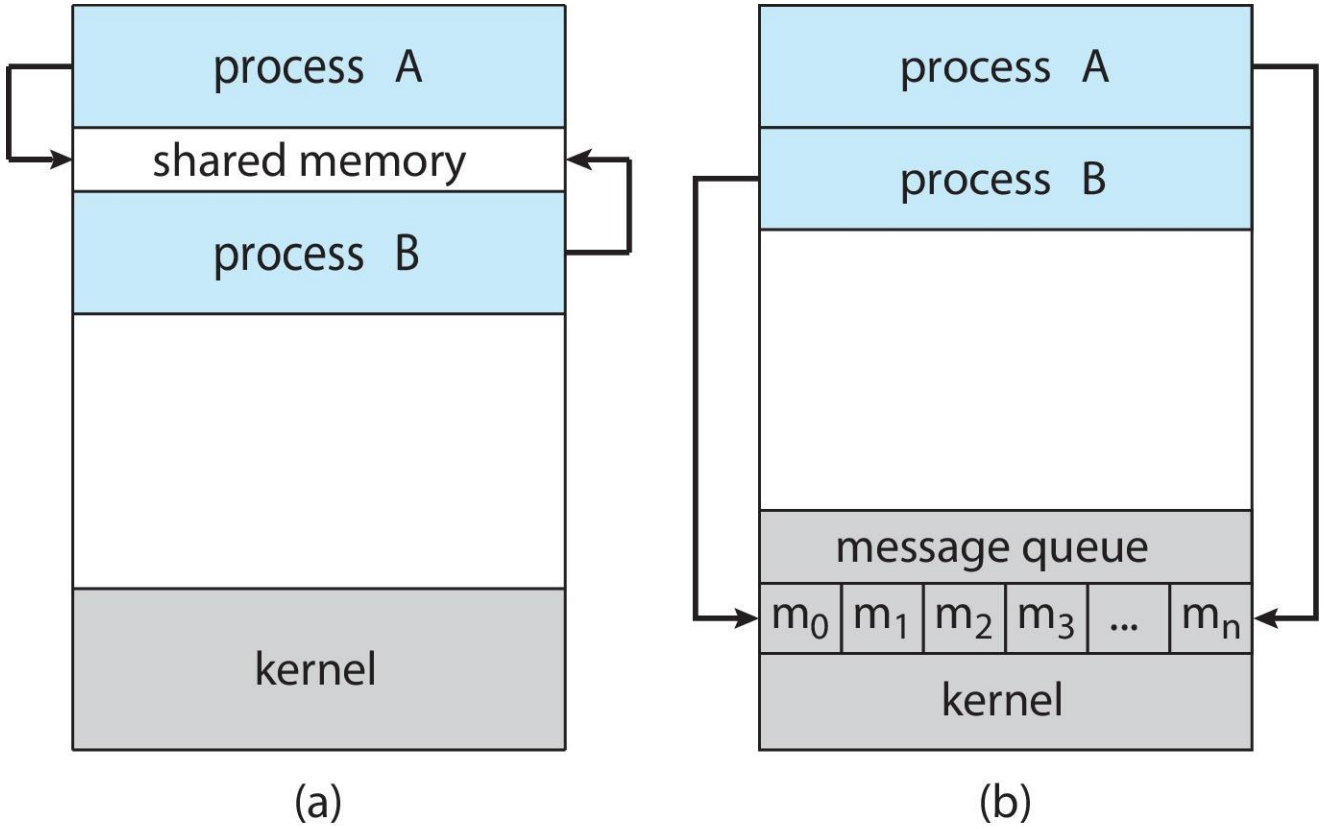
2. Message Passing

- Processes exchange messages via the **operating system**.
- Easier to implement in distributed systems.
- Useful when processes do not share memory.
- Example: Client-server communication via sockets.

Communications Models

(a) Shared memory.

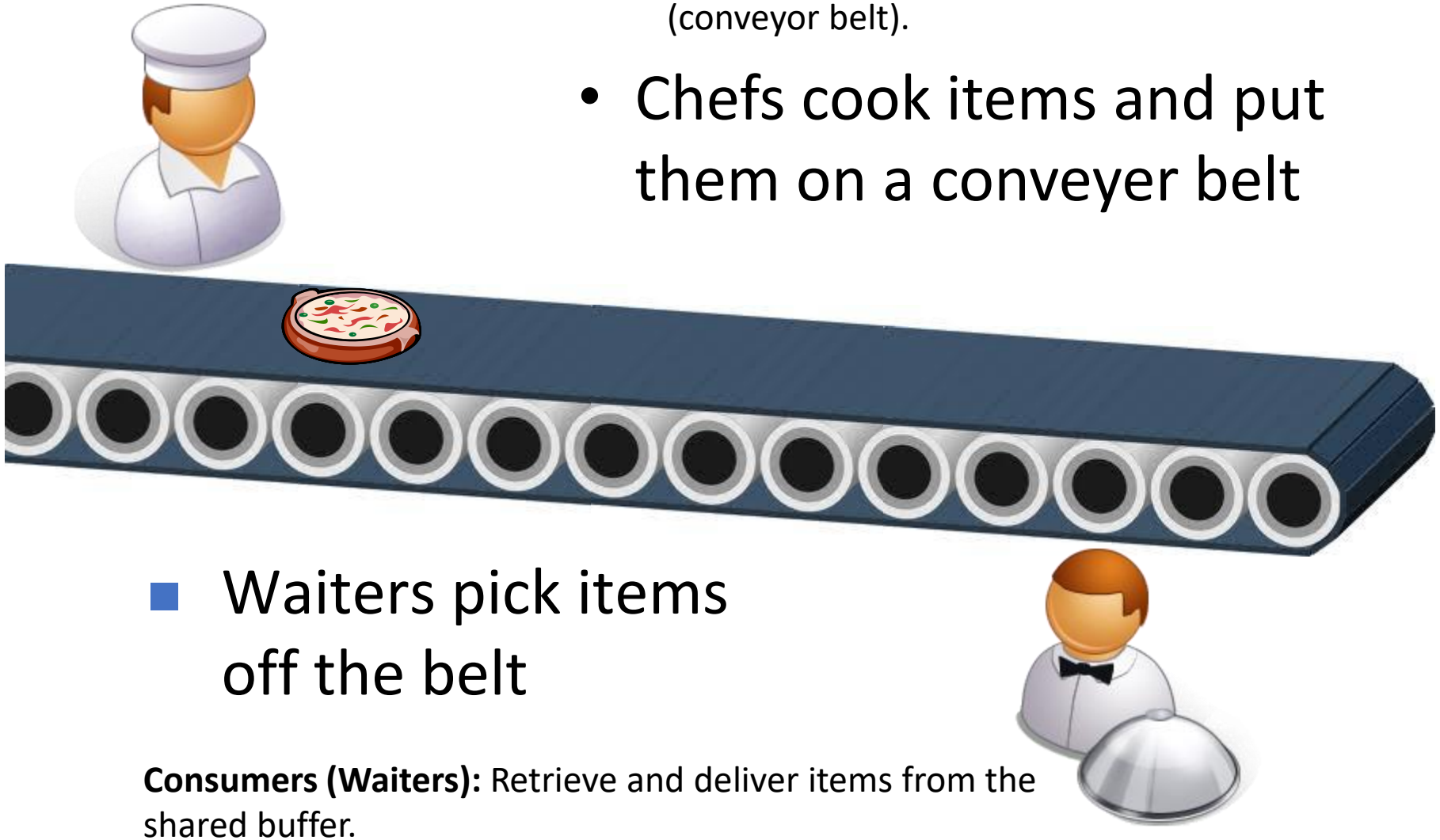
(b) Message passing.



Co-operating process

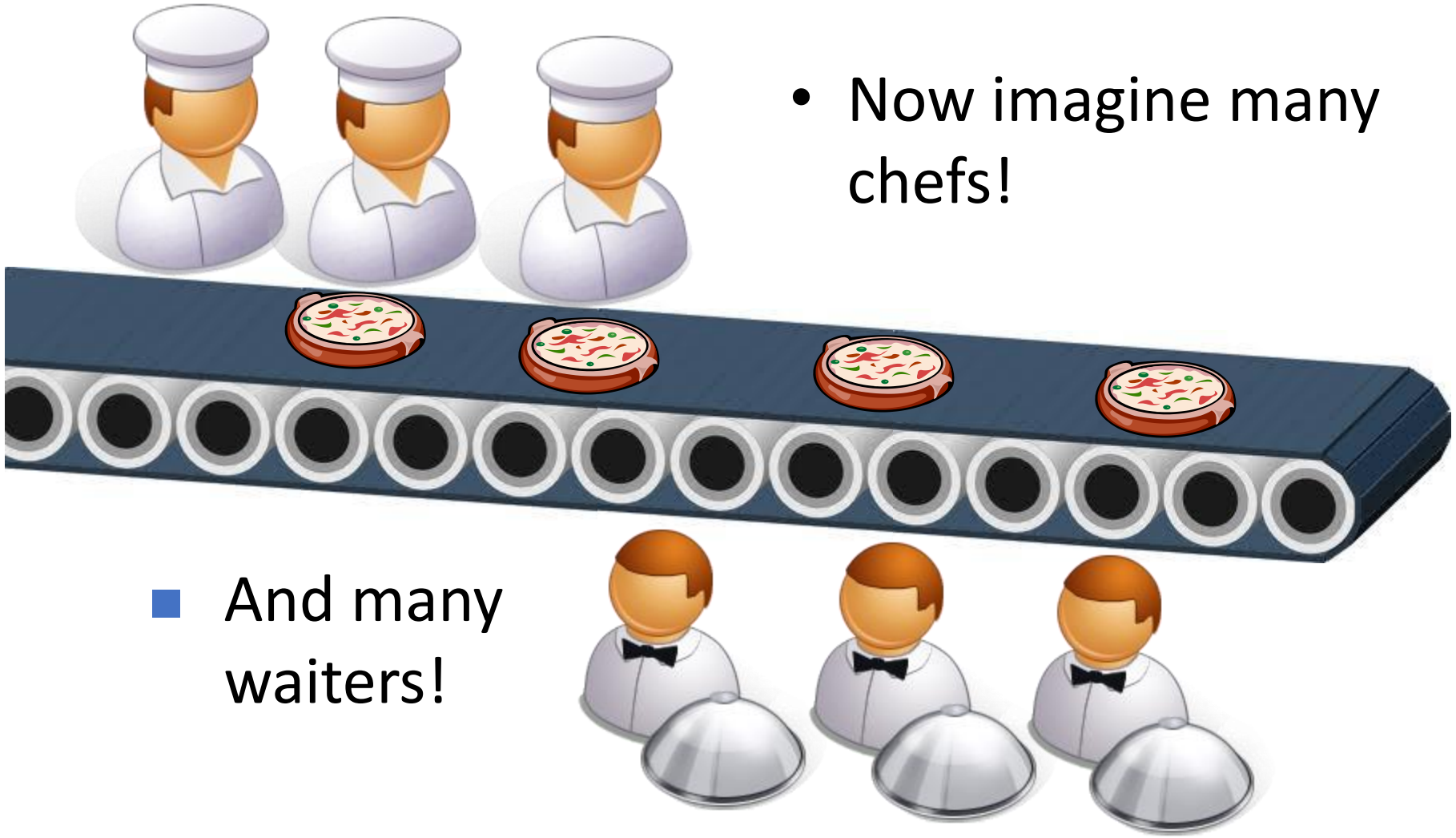
Producer - Consumer

Producers (Chefs): Prepare and place items onto a shared buffer (conveyor belt).



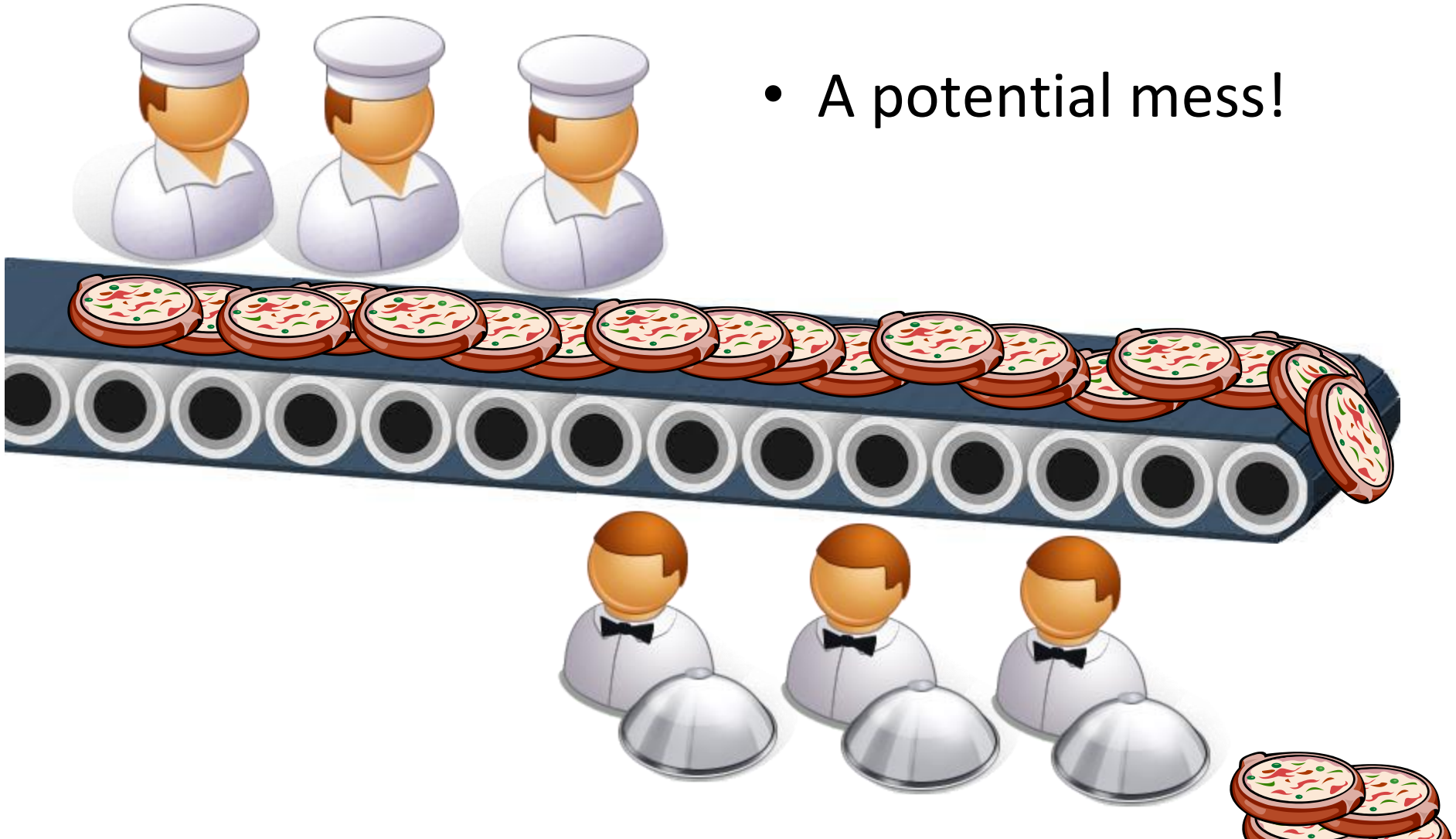
- Chefs cook items and put them on a conveyer belt
- Waiters pick items off the belt

Consumers (Waiters): Retrieve and deliver items from the shared buffer.



- Now imagine many chefs!
- And many waiters!

- A potential mess!



Producer-Consumer Problem



Chef = Producer
Waiter = Consumer



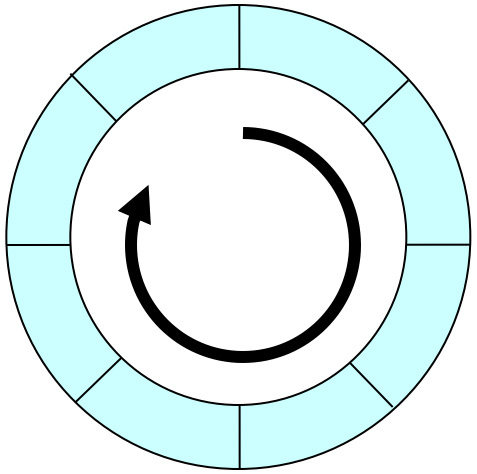
- Producers insert items
- Consumers remove items
- Shared resource: bounded buffer
  - Producers can't add items if the buffer is full, and consumers can't take items if it's empty. This prevents data loss and errors.
  - Efficient implementation: circular buffer with an insert and a removal pointer

Producer-Consumer



Chef = Producer
Waiter = Consumer



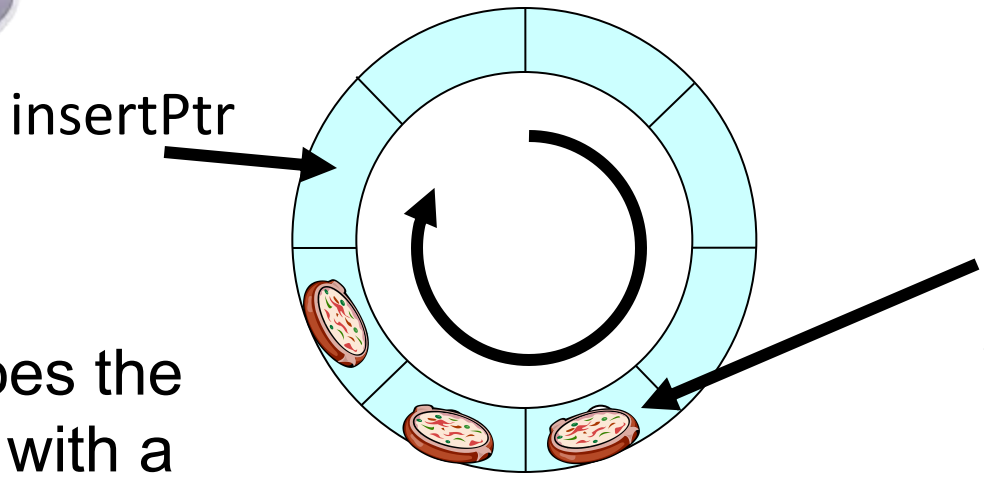


Producer-Consumer



Chef = Producer
Waiter = Consumer





What does the chef do with a new pizza?



Where does the waiter take a pizza from?

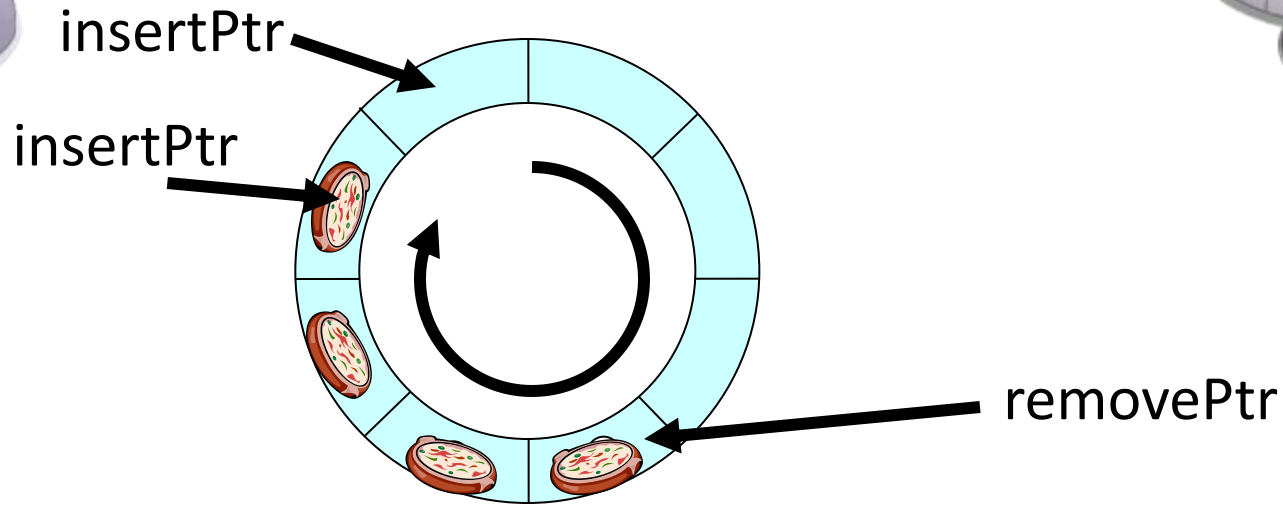


Producer-Consumer



Chef = Producer
Waiter = Consumer





Insert pizza

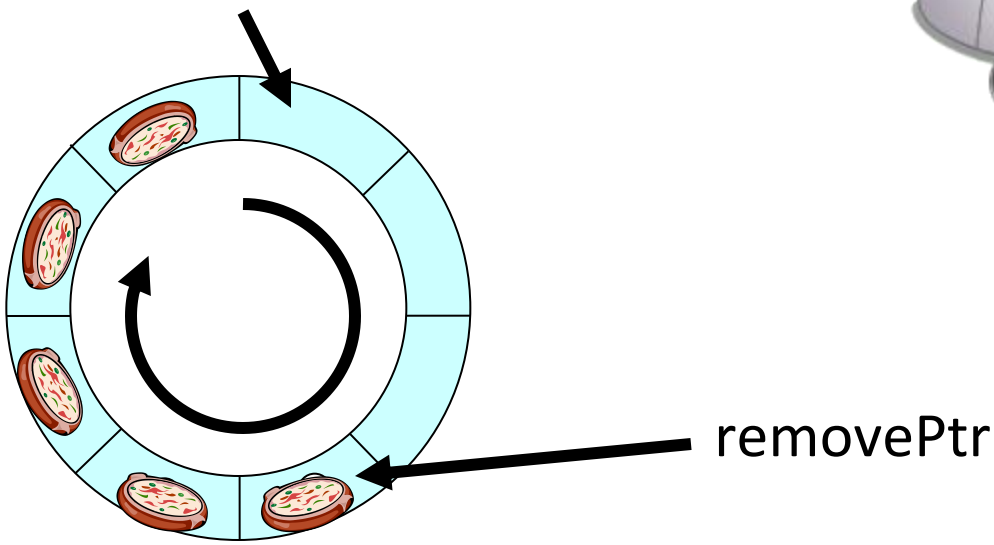


Producer-Consumer



Chef = Producer
Waiter = Consumer





Insert pizza

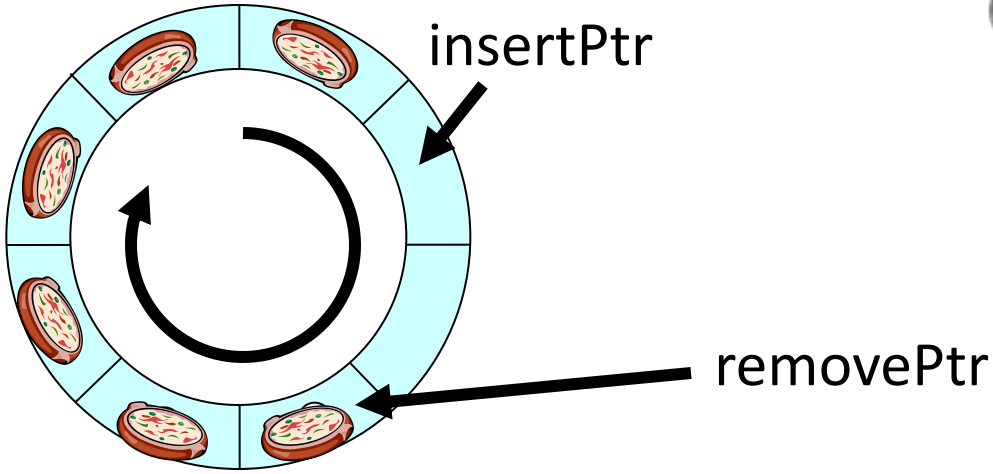


Producer-Consumer



Chef = Producer
Waiter = Consumer





Insert pizza

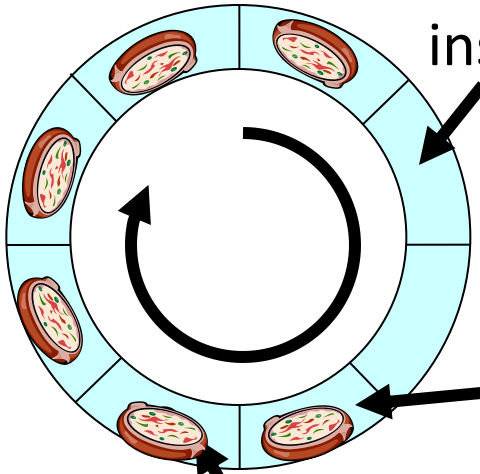


Producer-Consumer



Chef = Producer
Waiter = Consumer





Remove pizza

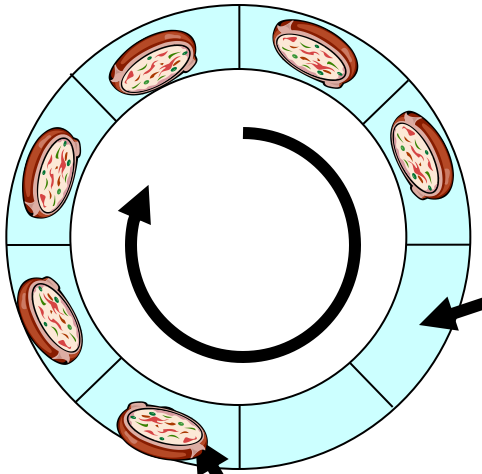


Producer-Consumer



Chef = Producer
Waiter = Consumer





insertPtr

removePtr

Insert pizza

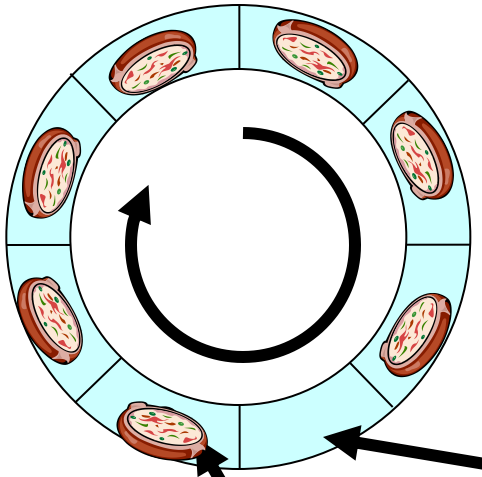


Producer-Consumer



Chef = Producer
Waiter = Consumer





insertPtr

removePtr

Insert pizza



Producer-Consumer

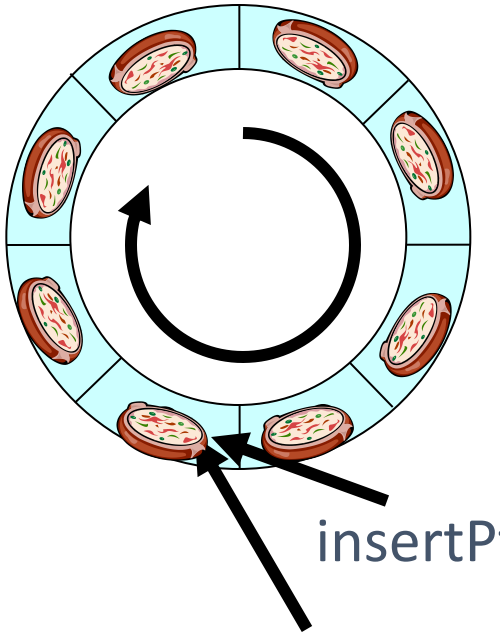


Chef = Producer

Waiter = Consumer



BUFFER FULL:
Producer must be blocked!



Insert pizza



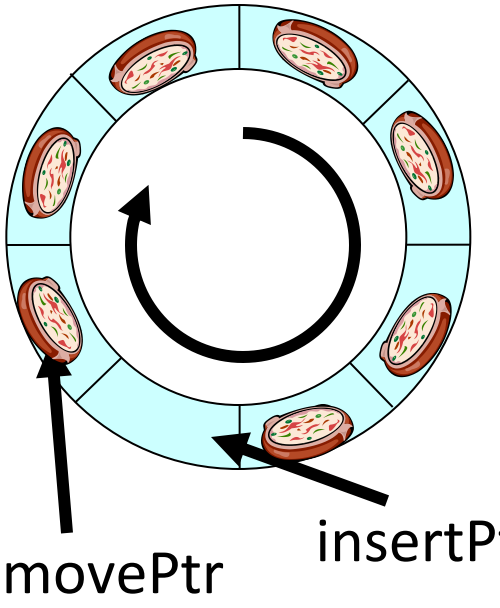


Producer-Consumer



Chef = Producer
Waiter = Consumer





Remove pizza

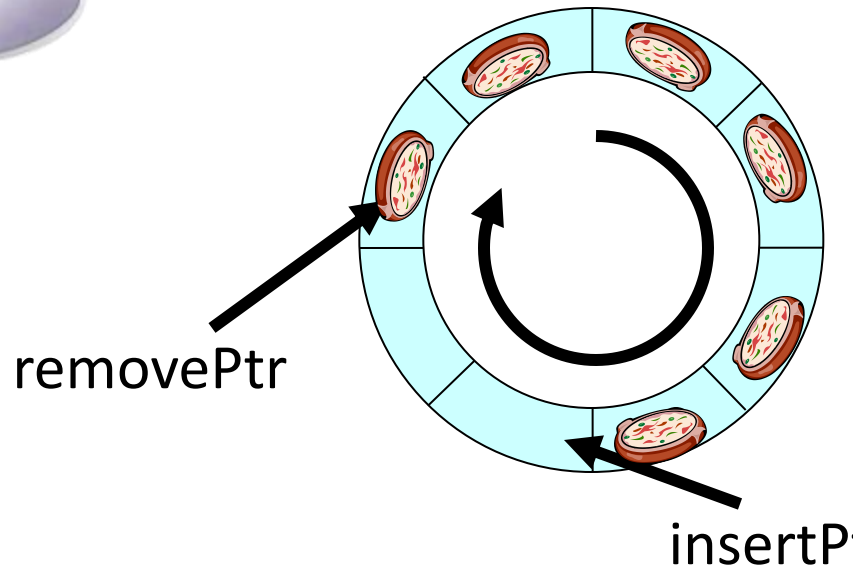


Producer-Consumer



Chef = Producer
Waiter = Consumer





Remove pizza

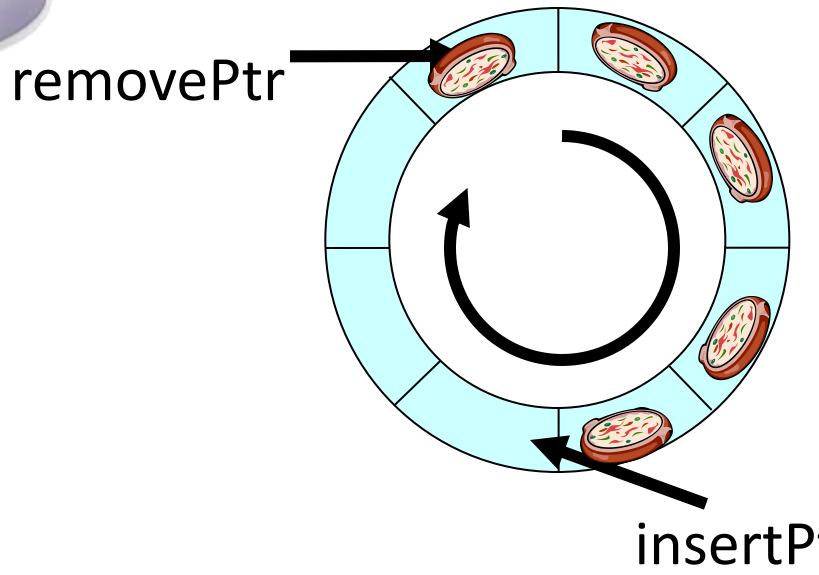


Producer-Consumer



Chef = Producer
Waiter = Consumer





Remove pizza



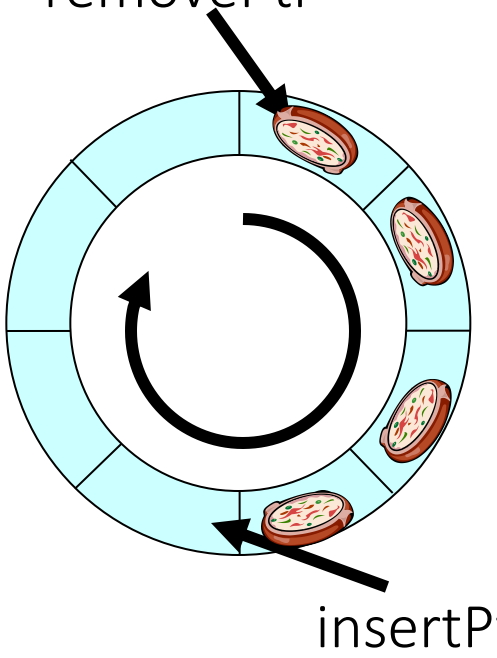
Producer-Consumer



Chef = Producer

Waiter = Consumer





Remove pizza

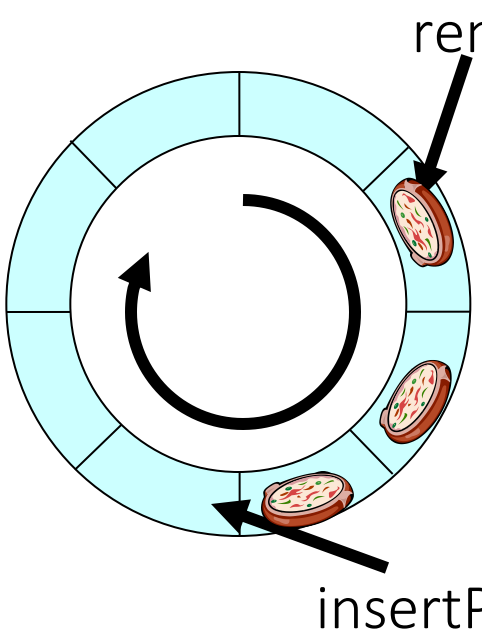


Producer-Consumer



Chef = Producer
Waiter = Consumer





Remove pizza

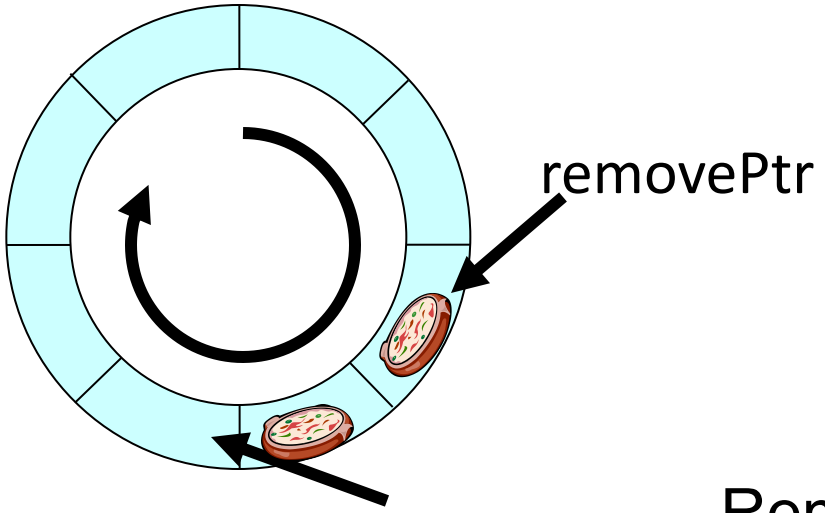


Producer-Consumer



Chef = Producer
Waiter = Consumer





Remove pizza

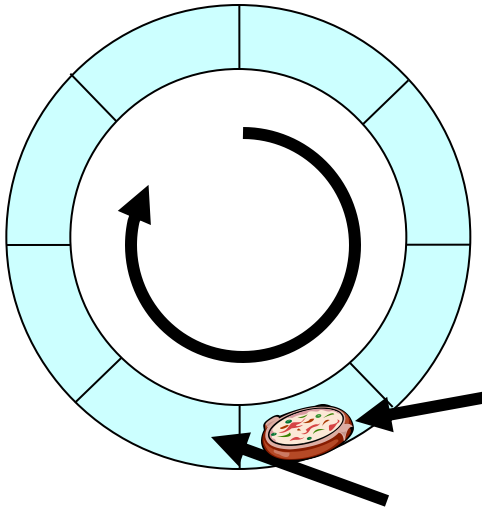


Producer-Consumer



Chef = Producer
Waiter = Consumer





Remove pizza



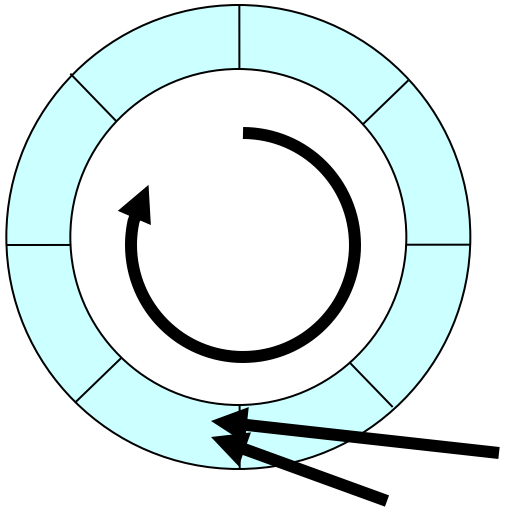
Producer-Consumer



Chef = Producer
Waiter = Consumer



BUFFER EMPTY:
Consumer must be blocked!



Remove pizza



Producer-Consumer Problem

- A classic synchronization problem in operating systems where multiple processes (producers and consumers) share a fixed-size buffer.
- **Producer**: Generates data/items and places them into the buffer.
- **Consumer**: Takes data/items from the buffer for processing.
- **Challenge**:
  - Ensure the **producer** does not add to a **full buffer**.
  - Ensure the **consumer** does not remove from an **empty buffer**.
- **Goal**:
  - Ensure proper coordination so data is neither lost nor duplicated.
  - Achieve proper synchronization to avoid race conditions and ensure smooth data flow



Solution for the Producer

- If the **buffer is full**, the producer must either:
  - **Wait (sleep)** until space becomes available, or
  - **Pause production** instead of discarding data.
- When the **consumer removes an item** from the buffer,
  - The consumer **notifies the producer**,
  - The producer **resumes filling** the buffer.

Solution for the Consumer

- If the **buffer is empty**, the consumer must:
  - **Wait (sleep)** until new data becomes available.
- When the **producer adds an item** to the buffer:
  - The producer **notifies the consumer**,
  - The consumer **wakes up** and resumes consuming data.

Producer-Consumer Problem

- Paradigm for cooperating processes:
  - ***producer*** process produces information that is consumed by a ***consumer*** process
- Key Concepts
  - **Producer**: Creates items and adds them to the buffer.
  - **Consumer**: Removes items from the buffer and processes them.
  - **Buffer**: Shared storage that holds items temporarily.

Two Variations:

1. **Unbounded Buffer**
    1. Buffer has *no practical size limit*
    2. **Producer**: never waits
    3. **Consumer**: waits if buffer is empty
2. **Bounded Buffer**
    1. Buffer has a *fixed size*
    2. **Producer**: waits if buffer is full
    3. **Consumer**: waits if buffer is empty

Shared Memory Solution

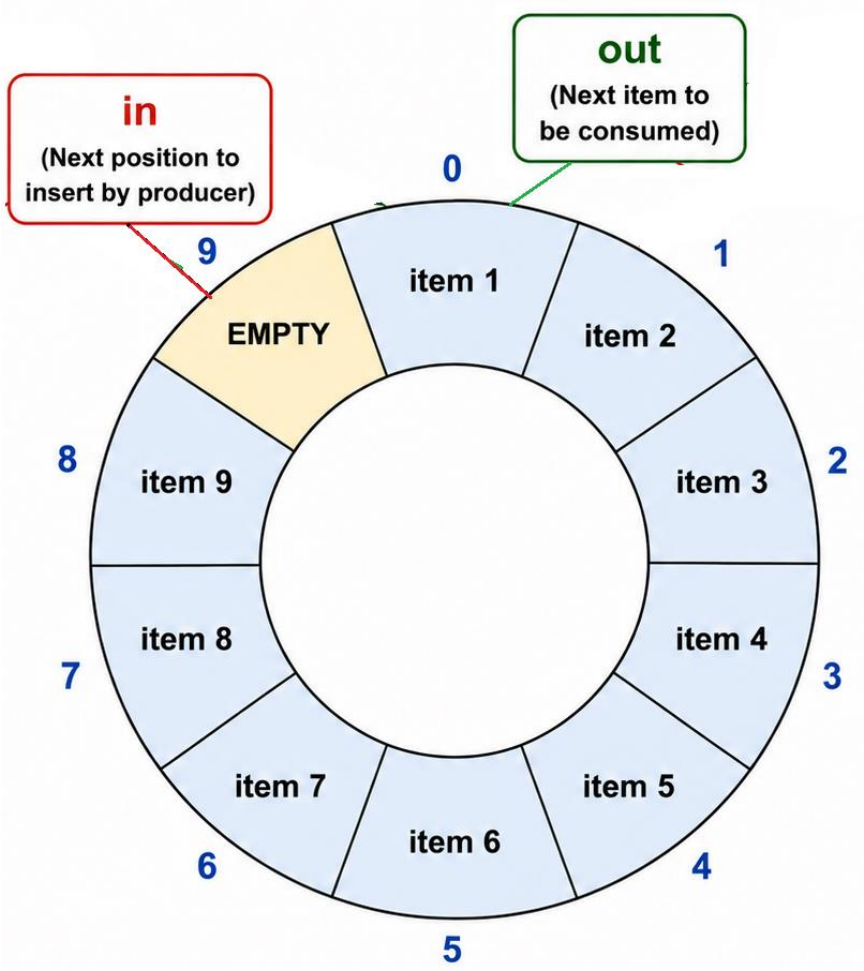
- A **shared memory area** is allocated for processes that wish to communicate.
- Communication is **controlled by user processes**, not the operating system.
- The **main challenge**:
  - Ensuring synchronization when multiple processes **access shared memory**.
  - Preventing race conditions and ensuring **data consistency**.

Bounded-Buffer – Shared Memory Solution

- Shared data

```
#define BUFFER_SIZE 10
typedef struct {
  . . .
} item;

item buffer[BUFFER_SIZE];
int in = 0; // index for producer
int out = 0; // index for consumer
```



- Solution presented in the figure is correct, but can only use **BUFFER_SIZE-1** items; that is: 9 items

✓ The circular buffer can hold at most **BUFFER_SIZE - 1** items.
✓ For BUFFER_SIZE = 10, the buffer can store **9 items** at a time.
✓ This ensures proper distinction between **full** and **empty** states.

Bounded-Buffer (Cont.)

In a **circular buffer** (ring buffer) implementation, we keep two indices:

- in → where the producer writes the next item
- out → where the consumer reads the next item

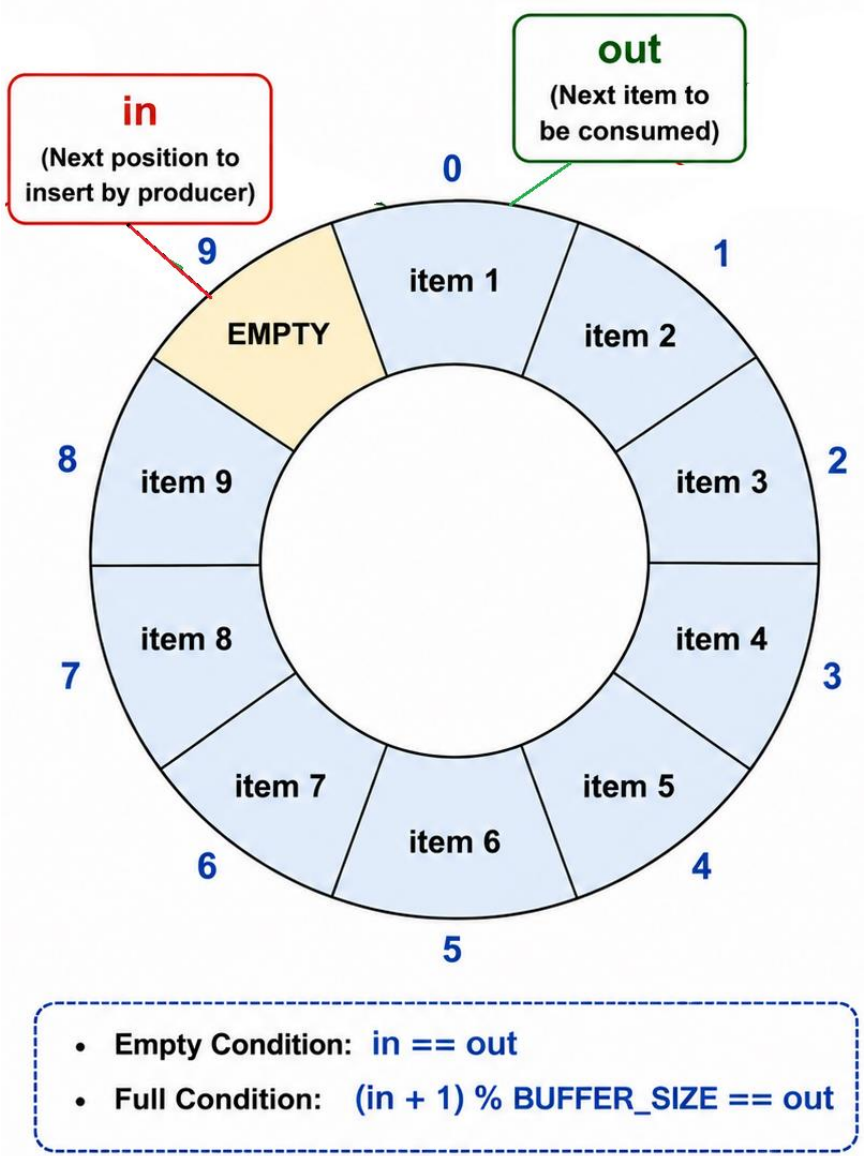
- **Producer** inserts at position in → (in + 1) % BUFFER_SIZE.
- **Consumer** removes from position out → (out + 1) % BUFFER_SIZE
- Waiting occurs when:
  - Producer finds buffer full.
  - Consumer finds buffer empty.

- **Empty Buffer:**
  Condition: in == out

[_][_][_][_][_][_][_][_][_][_]
↑out
↑in

- **Full Buffer:**
  Condition: (in + 1) % BUFFER_SIZE == out

[A][B][C][D][E][F][G][H][I][_]
↑out ↑in



Producer Code

```
int count = 0;
void producer( void )
{
 int itemP;
 while(1)
 {
  Produce_item(item P)
  while( count == n);
  buffer[ in ] = item P;
  in = (in + 1)mod n
  count = count + 1;
 }
}
```



```
Load Rp, m[count]
Increment Rp
Store m[count], Rp
```

Buffer[in] = item P → Buffer[5] = F. (F is inserted now)
in = (in + 1) mod n → (5 + 1)mod 8→ 6, therefore in = 6; (next empty buffer)
After insertion of F, Buffer looks like this
Where **out = 0**, but **in = 6**

0	A
1	B
2	C
3	D
4	E
5	F
6	
7	

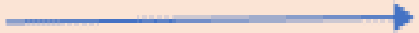
Buffer

```
  (pointer to the next empty space)→in    = 5
(pointer to the first filled space)→out   = 0
     (number of elements in Buffer)→count = 5
                  (Size of Buffer)→n      = 8
```

0	A
1	B
2	C
3	D
4	E
5	
6	
7	

Consumer Code

```
int count;
void consumer(void)
{
 int itemC;
 while( 1 )
 {
  while( count == 0 );
  itemC = buffer[ out ];
  out = ( out + 1 ) mod n;
  count = count -1;
 }
}
```



```
Load Rc, m[count]
Decrement Rc
Store m[count], Rc
```

itemC = Buffer[out]→ itemC = A (since out is 0)
out = (out + 1) mod n → (0 + 1)mod 8→ 1, therefore
out = 1(first filled position)
A is removed now
After removal of A, Buffer look like this
Where **out = 1**, and **in = 6**

0	
1	B
2	C
3	D
4	E
5	F
6	
7	

Producer Process – Shared Memory

```
item next_produced; // temporary variable to hold the item being produced

while (true) {     /* produce an item in next produced */
  while (((in + 1) % BUFFER_SIZE) == out) // Check if the buffer is full
        ; /* do nothing */
  buffer[in] = next_produced; // Place the item into the buffer
  in = (in + 1) % BUFFER_SIZE; // Update 'in' to the next position in circular buffer
}
```

Consumer Process – Shared Memory

```
item next_consumed;

while (true) {
      while (in == out) // Check if the buffer is full
            ; /* do nothing */
      next_consumed = buffer[out]; // Consume the item from the buffer
      out = (out + 1) % BUFFER_SIZE; // Update 'out' index in a circular manner
      /* consume the item in next consumed */
}
```

What about Filling all the Buffers?

- To solve the producer-consumer problem with **all buffers filled**, we can introduce an integer **counter**.
- **counter** keeps track of how many buffers are currently **full**.
- **Initialization:** counter = 0
- **Producer action:** After producing and placing an item in the buffer, **increment counter**.
- **Consumer action:** After consuming and removing an item from the buffer, **decrement counter**.
- This way:
  - counter == BUFFER_SIZE → Buffer is **full** (producer must wait).
  - counter == 0 → Buffer is **empty** (consumer must wait).

Producer

```
while (true) {
        /* produce an item in next produced */

        while (counter == BUFFER_SIZE)
                ; /* do nothing */
        buffer[in] = next_produced;
        in = (in + 1) % BUFFER_SIZE;
        counter++;
}
```

Consumer

```
while (true) {
        while (counter == 0)
                ; /* do nothing */
        next_consumed = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
      counter--;
        /* consume the item in next consumed */
}
```

IPC – Message Passing

- Processes communicate by **exchanging messages**, without using shared variables.
- Basic Operations:
  - **`send`**(*message*) → delivers a message to another process.
  - **`receive`**(*message*) → receives a message from another process.
- **Message Size:**
  - Can be **fixed** (easier to implement, less flexible).
  - Or **variable** (more flexible, but adds complexity).
- **Advantages:**
  - No need for shared memory.
  - Safer and simpler synchronization between processes.

Message Passing (Cont.)

- If processes *P* and *Q* wish to communicate, they need to:
  - Establish a **communication link** between them
  - Exchange messages via send/receive operation

📌 **Key Implementation Issues:**

- **Link Creation**
  - How are links established?
  - Can a link connect **more than two processes**?
- **Link Multiplicity**
  - How many links can exist between a pair of processes?
- **Link Capacity**
  - How many messages can the link hold?
- **Message Size**
  - Fixed size vs. variable size messages.
- **Directionality**
  - Is the link **unidirectional** (one-way) or **bidirectional** (two-way)?

Implementation of Communication Link

📌 Physical Implementation

- **Shared Memory** → processes communicate via common memory space
- **Hardware Bus** → communication over system bus
- **Network** → distributed communication across machines

📌 Logical Implementation

- **Direct vs. Indirect** → processes communicate directly or via mailboxes/ports
- **Synchronous vs. Asynchronous** → blocking (waits for response) vs. non-blocking
- **Buffering** → automatic buffering vs. explicit management by the programme

Direct Communication

📌 **Key Idea**

- Processes must explicitly **name each other** to communicate.

Operations:

- send(P, message) → send message to process **P**
- receive(Q, message) → receive message from process **Q**

📌 **Properties of Communication Link**

- ✅ Link is established **automatically**
- ✅ Each link is tied to **exactly one pair** of processes
- ✅ Between each pair → **only one link** exists
- ✅ Links are usually **bi-directional** (can also be unidirectional)

Indirect Communication

📌 Key Idea

- Messages are sent and received via **mailboxes (ports)**.
- Each mailbox has a **unique ID**.
- Processes can communicate **only if they share a mailbox**

📌 Properties of Communication Link

- ✅ A link is created **only if a common mailbox exists**.
- ✅ A mailbox can be associated with **multiple processes**.
- ✅ A process pair may share **several communication links**.
- ✅ Links can be **unidirectional or bi-directional**.

Indirect Communication (Cont.)

📌 **Operations:**

- ✨ **Create** a new mailbox (port)
- ✨ **Send** and **receive** messages through the mailbox
- ✨ **Delete** a mailbox

📌 **Communication Primitives:**

- **Send(A, message)** → send a message to mailbox **A**
- **Receive(A, message)** → receive a message from mailbox **A**

Indirect Communication (Cont.)

Mailbox Sharing :

📌 **Problem:**

- Processes **P1, P2, and P3** share mailbox **A**
- **P1 sends**, but both **P2** and **P3** may receive
- ❓ Who gets the message?

📌 **Possible Solutions:**

- Restrict: one mailbox ↔ at most **two processes**
- Restrict: only **one process** can execute receive() at a time
- Flexible: system **arbitrarily selects** the receiver → sender is **notified**



Which of the following is the fastest IPC mechanism in most operating systems?

A) Pipes

C) Message Passing

B) Signals

D) Shared Memory

Next

In the Producer–Consumer problem, the bounded buffer is mainly used to:

A) Prevent memory leaks

C) Speed up the producer

B) Eliminate synchronization

D) Prevent buffer overflow and underflow

Next

The “convoy effect” in CPU scheduling happens when:

A) A short process executes before a long process

C) All processes arrive at the same time

B) CPU remains idle for a long time

D) A long process delays the execution of short processes

Next

Which synchronization tool is most commonly used in solving the Producer–Consumer problem?

A) Paging

C) Deadlock detection

B) Virtual memory

D) Semaphores

Next

Which synchronization primitive is commonly used to avoid race conditions in IPC?

A) Deadlock

C) Paging

B) Thrashing

D) Semaphore

Next

If the producer tries to add data when the buffer is full, it must:

A) Terminate

C) Wait until a consumer removes an item

B) Create a new buffer

D) Overwrite old data

Well done!

Thanks